

HistoricoAPI

Documentação técnica do microsserviço de gerenciamento do histórico de consultas, com interface GraphQL

DISCIPLINA

Arquitetura e Desenvolvimento em Java

TURMA E GRUPO

Turma 12ADJT, Grupo 46

INTEGRANTES

Gustavo Chauar Correa (RM 370978)
Caroline Da Silva Brunheti (RM 372744)
Davi Faustino Souza Lima (RM 373019)
Eduardo Mendes Germano (RM 371760)
Roberto Domingos Locatelli (RM 370928)

REPOSITÓRIO

<https://github.com/JavaPosTech/HistoricoAPI>

ENTREGA E VERSÃO

Setembro de 2026, versão 1.0.0

Sumário

Introdução	3
Descrição do problema	3
Objetivo do projeto	4
Arquitetura do Sistema	5
Descrição da arquitetura	5
Diagrama da arquitetura	7
Descrição dos Endpoints da API	9
Tabela de endpoints	9
Exemplos de requisição e resposta	10
Configuração do Projeto	13
Configuração do Docker Compose	13
Instruções para execução local	14
Qualidade do Código	16
Boas práticas utilizadas	16
Repositório do Código	18
URL do repositório	18
Organização do repositório	18

Introdução

Descrição do problema

Em um ambiente hospitalar, é essencial contar com sistemas que garantam o **agendamento eficaz de consultas**, o **gerenciamento do histórico de pacientes** e o **envio de lembretes automáticos** para garantir a presença dos pacientes nas consultas. Este sistema deve ser acessível a diferentes tipos de usuários (médicos, enfermeiros e pacientes), com acesso controlado e funcionalidades específicas para cada perfil.

Para atender a esse cenário, a solução da fase é decomposta em microsserviços. O atendimento gera informação em pontos distintos do fluxo: o cadastro do paciente, o agendamento das consultas e o registro do histórico clínico. Espalhada por serviços diferentes, essa informação obriga o profissional de saúde a várias consultas até montar o quadro completo de um paciente antes do atendimento.

A **HistoricoAPI** atende exatamente esse recorte: consolidar, em uma única consulta, os dados cadastrais do paciente, o seu histórico clínico e as consultas agendadas. Os usuários são os profissionais de saúde e os demais serviços da fase que precisam recuperar esse conjunto de dados, nunca alterá-lo.

Por ser uma API de **leitura**, o desafio central não é a modelagem de escrita, e sim expor um agregado profundo sem obrigar o cliente a encadear chamadas ou a receber mais dados do que precisa. Daí a escolha de uma interface **GraphQL**, na qual o próprio cliente declara os campos desejados.

Recorte desta API dentro da solução. *O agendamento das consultas e a carga inicial do banco são responsabilidade da AgendamentoAPI, que também é dona das migrations do schema compartilhado. A HistoricoAPI consome esse mesmo banco e responde exclusivamente pela consulta do histórico.*

Objetivo do projeto

O objetivo é desenvolver um **backend simplificado e modular**, com foco em **segurança** e **comunicação assíncrona**, garantindo que o sistema seja escalável, seguro e que utilize boas práticas de autenticação, autorização e comunicação entre serviços.

No recorte desta API, isso se traduz em disponibilizar o histórico de consultas dos pacientes por uma interface GraphQL, integrada ao banco de dados compartilhado pelos microserviços da fase e empacotada em contêiner, de modo que o serviço possa escalar e ser substituído de forma independente dos demais.

Objetivos específicos

- **Expor o histórico por GraphQL:** disponibilizar a query `getHistoricoPaciente`, que devolve dados cadastrais, histórico clínico e consultas agendadas em uma única requisição.
- **Integrar-se ao banco compartilhado:** ler o schema mantido pelas migrations da AgendamentoAPI, sem duplicar a responsabilidade de evoluir as tabelas.
- **Manter o serviço modular:** isolar a leitura do histórico em um microserviço próprio, com o seu ciclo de build, imagem e implantação independentes.
- **Tratar os erros de forma previsível:** devolver `BAD_REQUEST` e `NOT_FOUND` com mensagens claras, sem vaziar detalhes internos do banco ou da aplicação.
- **Documentar a API:** publicar Swagger UI e GraphiQL, permitindo explorar e executar a consulta sem ferramenta externa.
- **Garantir a qualidade por testes:** manter uma suíte de integração real contra PostgreSQL, executada a cada Pull Request.
- **Empacotar e orquestrar:** entregar a aplicação em imagem Docker, subindo junto ao banco por Docker Compose.

Como o resultado é verificado

A API sobe por Docker Compose junto ao PostgreSQL compartilhado e responde à query `getHistoricoPaciente` pela GraphiQL ou pela Swagger UI, ambas publicadas pela própria aplicação. A suíte de integração roda no build (`./gradlew build`), executado pela CI a cada Pull Request para a branch `main`.

Arquitetura do Sistema

Descrição da arquitetura

A aplicação segue a **arquitetura em camadas** do ecossistema Spring, com direção de dependência única: cada camada conhece apenas a imediatamente abaixo. O código é organizado **por camada e, dentro de cada camada, por domínio**, então a estrutura de pacotes revela a responsabilidade técnica e o assunto de negócio.

Camadas da aplicação

Camada	Responsabilidade	Pacote
Controller	Resolver GraphQL: recebe o argumento da query e delega ao serviço	<code>...historicoapi.controller</code>
Service	Validação, orquestração dos repositórios e montagem do DTO	<code>...historicoapi.service</code>
Repository	Acesso a dados declarativo, por interfaces do Spring Data JPA	<code>...historicoapi.repository</code>
Model	Entidades JPA e DTOs de saída (records imutáveis)	<code>...historicoapi.model</code>
Exceptions	Exceções de negócio e tratamento centralizado de erros	<code>...historicoapi.exceptions</code>

Tecnologias utilizadas

Item	Tecnologia	Versão
Linguagem	Java	21
Framework	Spring Boot	4.0.5
Interface da API	Spring for GraphQL + Spring MVC	—
Persistência	Spring Data JPA / Hibernate	—
Banco de dados	PostgreSQL	18
Build	Gradle (Kotlin DSL)	9.7.1
Logging e documentação	Log4j2 (via SLF4J) e SpringDoc OpenAPI	3.1.0
Testes	JUnit 5, JaCoCo e Flyway (perfil de teste)	—
Containerização	Docker / Docker Compose	—

Estrutura de pastas

```
src/
├── main/
│   ├── java/br/com/fiap/historicoapi/
│   │   ├── config/           # DataBaseConfig (dev, prod) e SwaggerConfig
│   │   ├── controller/      # HistoricoController: resolver GraphQL
│   │   ├── service/         # HistoricoService: regras de negócio
│   │   ├── repository/      # paciente, agendamento, historicopaciente
│   │   ├── model/
│   │   │   ├── entity/      # entidades JPA
│   │   │   └── dto/         # records de saída
│   │   ├── exceptions/      # exceções, DTOs de erro e handler global
│   │   └── util/            # FormatadorData
│   └── resources/
│       ├── application.yaml # perfis dev, prod e test
│       ├── log4j2.xml
│       └── graphql/         # query.graphqls e schema.graphqls
└── test/
    ├── java/...             # testes de integração
    └── resources/db/migration/ # migrations do perfil test
```

Decisões de arquitetura

- **Interface GraphQL em vez de REST.** O histórico é um agregado profundo (paciente + histórico clínico + consultas). Com GraphQL o cliente escolhe exatamente os campos desejados, em uma única requisição, eliminando tanto o *over-fetching* quanto chamadas encadeadas. Todo o tráfego passa por um endpoint único.
- **PostgreSQL 18 como banco relacional.** O domínio é fortemente relacional (paciente, histórico, agendamento e médico) e exige integridade referencial. O banco é **compartilhado** pelos microsserviços da fase, garantindo uma fonte única de verdade.
- **Separação entre dono e leitor do schema.** As migrations Flyway pertencem à AgendamentoAPI, que cria as tabelas e a carga inicial. Esta API não possui `ddl-auto` e apenas mapeia o schema existente, de modo que não há dois serviços disputando a evolução do banco.
- **Carregamento explícito de relacionamentos.** Com `spring.jpa.open-in-view: false` não existe sessão JPA aberta na camada web; toda associação necessária à resposta é carregada dentro da transação, por `@EntityGraph` no próprio repositório.
- **Containerização.** Build multi-stage no Dockerfile, no qual `gradle:jdk21-alpine` compila e `eclipse-temurin:21-jre-alpine` executa, mais a orquestração por Docker Compose sobre uma rede compartilhada, tornando a subida do ambiente reproduzível.

Diagrama da arquitetura

A figura abaixo mostra o caminho percorrido por uma requisição, da ferramenta cliente até o banco de dados, e os componentes transversais que atuam em todas as camadas.

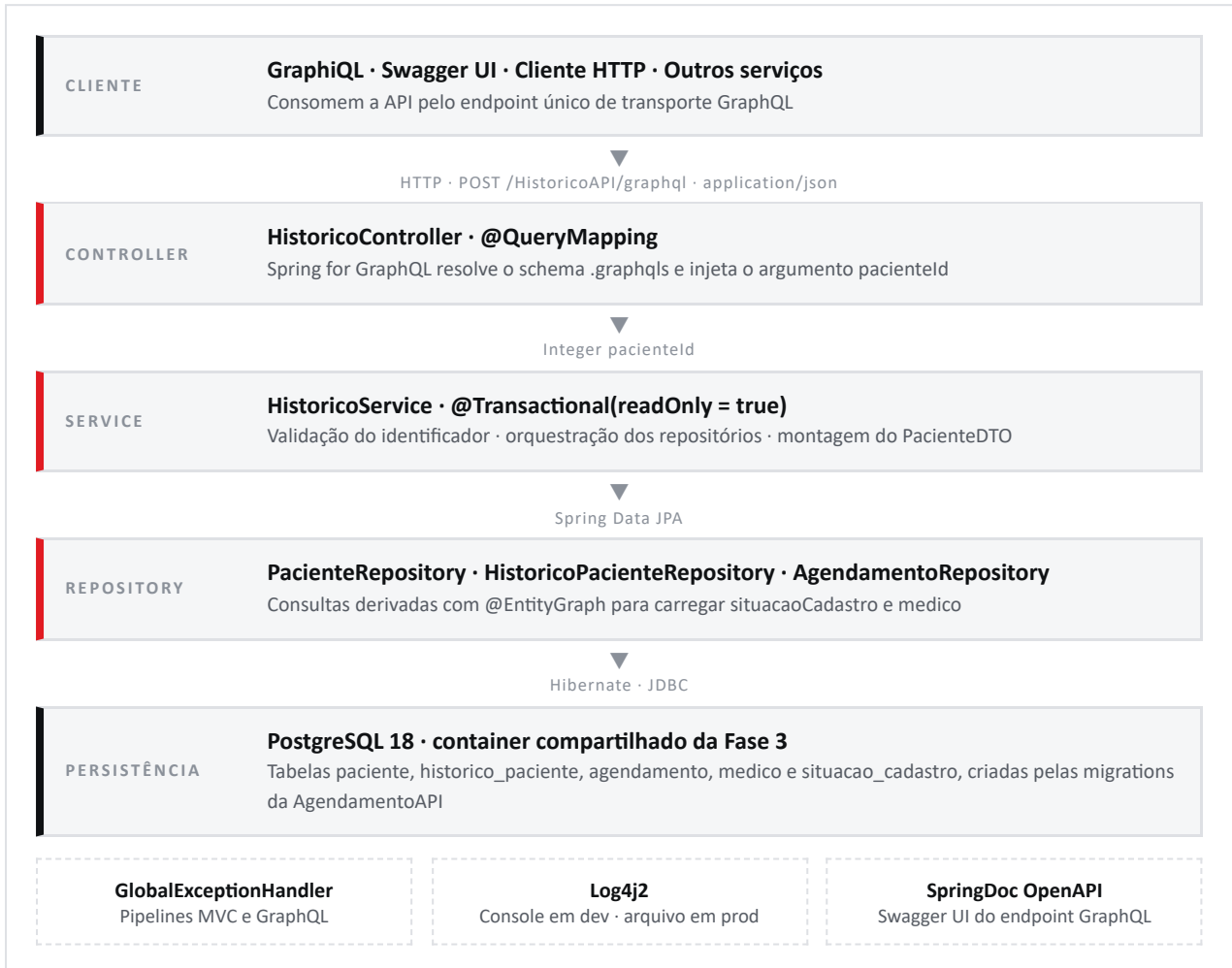


Figura 1. Fluxo de uma requisição através das camadas da aplicação

Fluxo completo de uma consulta. O cliente envia a query, o controller recebe *pacienteId*, o service valida o valor e busca o paciente, o histórico clínico e as consultas, os DTOs convertem as entidades e formatam as datas, e a resposta volta em JSON no envelope `{ data, errors }` da especificação GraphQL.

Topologia de execução em contêineres

Os microsserviços da fase compartilham um único PostgreSQL. O banco sobe de forma independente e cria a rede `shared-net`; cada serviço entra nela como rede externa.

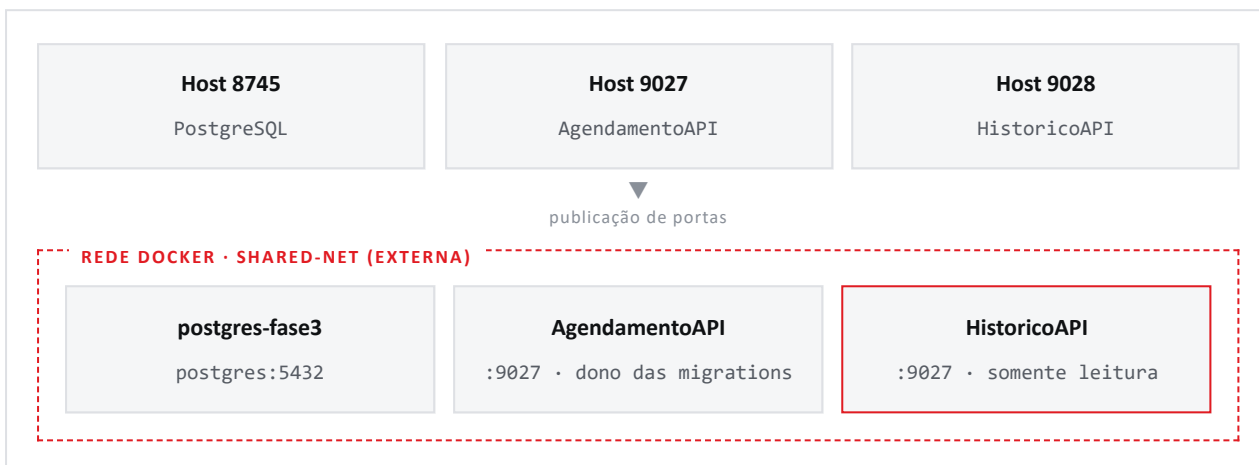


Figura 2. Contêineres e rede compartilhada da Fase 3

Dentro da rede o banco é sempre alcançado pelo hostname `postgres` na porta interna `5432`. A porta `8745` é apenas a exposição no host, para acesso por IDE ou cliente SQL. Como todas as APIs escutam em `9027` dentro do próprio contêiner, elas se diferenciam pela porta publicada: a AgendamentoAPI em `9027` e a HistoricoAPI em `9028`.

Ordem de inicialização. Os arquivos *Compose* do PostgreSQL **criam** a rede `shared-net`; o arquivo da API a **consome** como rede externa. Por isso o banco precisa subir primeiro: iniciar apenas o *Compose* da API resulta em erro de rede inexistente.

Descrição dos Endpoints da API

Tabela de endpoints

Por ser uma API GraphQL, existe um **único endpoint de dados**: a operação é descrita no corpo da requisição, e não na URL. Todas as rotas são prefixadas pelo context path `/HistoricoAPI`, que diferencia maiúsculas de minúsculas.

Endpoint	Método	Descrição
<code>/HistoricoAPI/graphql</code>	POST	Endpoint único de dados. Recebe <code>query</code> , <code>operationName</code> e <code>variables</code> ; devolve o envelope <code>{ data, errors }</code> .
<code>/HistoricoAPI/graphiql</code>	GET	Interface interativa GraphiQL, para explorar o schema e executar consultas no navegador.
<code>/HistoricoAPI/swagger-ui/index.html</code>	GET	Documentação Swagger UI, com o endpoint GraphQL descrito e exemplos prontos.
<code>/HistoricoAPI/v3/api-docs</code>	GET	Documento OpenAPI bruto, em JSON, para importação em outras ferramentas.

Operações GraphQL disponíveis

Operação	Tipo	Argumento	Retorno
<code>getHistoricoPaciente</code>	Query	<code>pacienteId: ID!</code>	<code>Paciente</code> , com os dados cadastrais, a lista <code>historico</code> e a lista <code>consultas</code> .

Erros previstos

Situação	Classificação	Mensagem
Id nulo, zero ou negativo	<code>BAD_REQUEST</code>	O ID do Paciente deve ser um número inteiro positivo!
Paciente inexistente	<code>NOT_FOUND</code>	Paciente não encontrado - ID: <code>{id}</code>
Falha inesperada	<code>INTERNAL_ERROR</code>	Mensagem genérica; o detalhe fica apenas no log da aplicação.

Por que o status é sempre 200. Seguindo a especificação GraphQL, o transporte HTTP responde 200 OK mesmo quando a execução falha. O problema é sinalizado no array `errors`, com a classificação em `extensions.classification`.

Exemplos de requisição e resposta

Requisição: consulta do histórico

```
POST /HistoricoAPI/graphql HTTP/1.1
Host: localhost:9017
Content-Type: application/json

{
  "operationName": "BuscarHistoricoPaciente",
  "query": "query BuscarHistoricoPaciente($pacienteId: ID!) { getHistoricoPaciente(pacienteId: $pacienteId) { id nome sobrenome cpf email telefone endereco dataNascimento dataCadastro situacaoCadastro historico { id queixaPrincipal historicoDoenca medicamentos alergias observacoes } consultas { id nomeMedico dataHoraConsulta observacao dataCadastro } } }",
  "variables": { "pacienteId": 1 }
}
```

A mesma query, formatada

```
query BuscarHistoricoPaciente($pacienteId: ID!) {
  getHistoricoPaciente(pacienteId: $pacienteId) {
    id
    nome
    sobrenome
    cpf
    email
    telefone
    endereco
    dataNascimento
    dataCadastro
    situacaoCadastro
    historico { id queixaPrincipal historicoDoenca medicamentos alergias observacoes }
    consultas { id nomeMedico dataHoraConsulta observacao dataCadastro }
  }
}
```

Formato dos dados. Campos declarados como *ID!* trafegam como *String* por definição da especificação GraphQL. As datas já saem formatadas pelo utilitário *FormatadorData*, nos padrões *dd/MM/yyyy* e *dd/MM/yyyy - HH:mm:ss*.

Resposta 200 OK: paciente encontrado

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "data": {
    "getHistoricoPaciente": {
      "id": "1",
      "nome": "PEDRO",
      "sobrenome": "ALMEIDA",
      "cpf": "12345678901",
      "email": "pedro.almeida@email.com",
      "telefone": "(19) 99999-1001",
      "endereco": "Rua das Palmeiras, 50 - Limeira - SP",
      "dataNascimento": "15/05/1990",
      "dataCadastro": "10/01/2026 - 14:32:05",
      "situacaoCadastro": "ATIVO",
      "historico": [
        {
          "id": "1",
          "queixaPrincipal": "Dor no peito",
          "historicoDoenca": "Paciente relata dores no peito recorrentes.",
          "medicamentos": "Losartana 50mg",
          "alergias": "Nenhuma alergia conhecida.",
          "observacoes": "Recomendada avaliação cardiológica."
        }
      ],
      "consultas": [
        {
          "id": "1",
          "nomeMedico": "JOAO",
          "dataHoraConsulta": "22/08/2026 - 08:00:00",
          "observacao": "Primeira consulta cardiológica.",
          "dataCadastro": "10/01/2026 - 14:35:12"
        },
        {
          "id": "4",
          "nomeMedico": "JOAO",
          "dataHoraConsulta": "22/08/2026 - 14:00:00",
          "observacao": "Retorno cardiológico.",
          "dataCadastro": "10/01/2026 - 14:36:40"
        }
      ]
    }
  }
}
```

Resposta: paciente inexistente

```
// variables: { "pacienteId": 999 }
{
  "errors": [
    {
      "message": "Paciente não encontrado - ID: 999",
      "path": [ "getHistoricoPaciente" ],
      "extensions": { "classification": "NOT_FOUND" }
    }
  ],
  "data": { "getHistoricoPaciente": null }
}
```

Resposta: identificador inválido

```
// variables: { "pacienteId": 0 }
{
  "errors": [
    {
      "message": "O ID do Paciente deve ser um número inteiro positivo!",
      "path": [ "getHistoricoPaciente" ],
      "extensions": { "classification": "BAD_REQUEST" }
    }
  ],
  "data": { "getHistoricoPaciente": null }
}
```

Resposta fora do pipeline GraphQL

Requisições que sequer chegam ao resolver, como uma URL inexistente, são tratadas pelo pipeline Spring MVC do `GlobalExceptionHandler`, que responde com o `ErrorResponseDTO` e o status HTTP correspondente, trazendo `status`, `title`, `instance`, `type`, `detail` e `timestamp`.

Códigos de retorno adotados

Código	Significado	Quando ocorre
200	OK	Toda execução GraphQL, com sucesso ou com erro sinalizado em <code>errors</code>
400	Bad Request	Corpo ilegível ou argumento inválido fora do pipeline GraphQL
404	Not Found	Rota inexistente no context path
500	Internal Server Error	Falha inesperada; o detalhe fica apenas no log

Configuração do Projeto

Configuração do Docker Compose

O projeto disponibiliza **três arquivos Compose**, com papéis distintos. A separação é intencional: o banco não pertence a nenhum serviço em particular, então subir a API não pode implicar subir um banco novo e isolado.

Arquivo	Papel na orquestração
<code>docker-compose-postgres-dev.yml</code>	PostgreSQL 18 de desenvolvimento, com credenciais fixas e sem depender do <code>.env</code> . Projeto <code>fiap-fase3-dev</code> , volume <code>fiap-postgres-data-dev</code> .
<code>docker-compose-postgres-prod.yml</code>	PostgreSQL 18 de produção: lê o <code>.env</code> , possui healthcheck com <code>pg_isready</code> e fuso <code>America/Sao_Paulo</code> . Projeto <code>fiap-fase3</code> , volume <code>fiap-postgres-data</code> .
<code>docker-compose-historicoapi.yml</code>	Apenas a API, construída pelo Dockerfile local, no perfil <code>prod</code> , publicando <code>9028:9027</code> e montando <code>./logs</code> . Declara <code>shared-net</code> como rede externa.

```

services:
  historicoapi:
    container_name: HistoricoAPI
    build: .
    image: historicoapi:1.0.0
    ports:
      - "9028:9027"          # host : container
    environment:
      TZ: America/Sao_Paulo
      SPRING_PROFILES_ACTIVE: prod
      DATABASE_IP: postgres   # hostname do container na rede
      DATABASE_PORT: 5432     # porta interna, nunca a 8745
      DATABASE_NAME: ${DATABASE_NAME:-postgres}
      DATABASE_USER: ${DATABASE_USER:-postgres}
      DATABASE_PASSWORD: ${DATABASE_PASSWORD:?defina no .env}
    volumes:
      - ./logs:/app/logs
    restart: always
    networks:
      - shared-net

networks:
  shared-net:
    external: true
  
```

- **Nomes fixos de projeto e volume.** Os dois Compose do banco são idênticos aos da AgendamentoAPI, então o container e os dados são sempre os mesmos, independentemente do repositório de onde o comando for executado.
- **Um único conjunto de variáveis.** `DATABASE_NAME`, `DATABASE_USER` e `DATABASE_PASSWORD` servem para criar o container do PostgreSQL e para a API se conectar a ele, de modo que as credenciais não têm como divergir.
- **Falha explícita em vez de banco inseguro.** A sintaxe `${DATABASE_PASSWORD:?...}` aborta o Compose com mensagem clara se a senha estiver vazia.

Instruções para execução local

Pré-requisitos: **Java 21** (a toolchain do Gradle resolve a versão automaticamente), **Docker** e **Docker Compose**, além do repositório da **AgendamentoAPI**, que executa as migrations e cria o schema.

1. Clonar o repositório do projeto
2. Subir o PostgreSQL compartilhado (uma única vez, a partir de qualquer projeto da fase)
3. Executar a AgendamentoAPI ao menos uma vez, para que as migrations criem o schema e a carga inicial
4. Executar a HistoricoAPI no perfil desejado e acessar a GraphQL ou a Swagger UI

```
# Desenvolvimento
git clone https://github.com/JavaPosTech/HistoricoAPI.git
cd HistoricoAPI
docker compose -f docker-compose-postgres-dev.yml up -d --wait
cd ../AgendamentoAPI && ./gradlew bootRun --args="--spring.profiles.active=dev"
cd ../HistoricoAPI && ./gradlew bootRun --args="--spring.profiles.active=dev"

# Produção (preencher antes o .env com DATABASE_NAME, USER e PASSWORD)
docker compose -f docker-compose-postgres-prod.yml up -d --wait
docker compose -f docker-compose-historicoapi.yml up -d

# Testes (basta o banco no ar; o Flyway do perfil test cria o schema)
./gradlew test
```

Windows. Utilize `.\gradlew.bat` no lugar de `./gradlew`. O IntelliJ traz as configurações prontas em `.run/` (BootRun DEV, BootRun PROD, Clean Build e Testes de Integração), que já injetam as variáveis `DATABASE_*`.

URLs e portas configuradas

Serviço	URL	Porta
API (perfil dev)	http://localhost:9017/HistoricoAPI	9017
API (perfil prod)	http://localhost:9028/HistoricoAPI	9028 (host) 9027 (container)
Endpoint GraphQL	POST /HistoricoAPI/graphql	9017 / 9028
GraphiQL	http://localhost:9017/HistoricoAPI/graphiql	9017
Swagger UI	http://localhost:9017/HistoricoAPI/swagger-ui/index.html	9017
Banco de dados	jdbc:postgresql://localhost:8745/postgres	8745 (host) 5432 (rede)

Perfis da aplicação

Perfil	Porta	Características
dev	9017	SQL formatado, Hibernate em DEBUG e binder em TRACE. Log apenas no console.
prod	9027	Log em arquivo rotativo diário em /app/logs , com expurgo após 30 dias.
test	—	Perfil da suíte de integração; único em que o Flyway está habilitado.

Qualidade do Código

Boas práticas utilizadas

O projeto adota as convenções idiomáticas do Spring Boot e princípios consolidados de projeto orientado a objetos. A suíte de integração serve como rede de segurança e é verificada a cada Pull Request: são **28 testes em 7 classes**, com **100% de cobertura** de instruções, linhas, métodos, classes e branches.

Princípios SOLID

- **Responsabilidade única.** Cada camada tem um único motivo para mudar: o controller expõe a operação GraphQL, o serviço concentra a regra de negócio, os repositórios cuidam do acesso a dados e os DTOs definem o contrato de saída. O próprio `HistoricoService` é decomposto em métodos privados curtos: `validarPacienteId`, `buscarPaciente`, `buscarHistoricoPaciente` e `buscarHistoricoConsultas`.
- **Inversão de dependência.** O serviço depende de interfaces `JpaRepository`, nunca de implementações concretas; quem as fornece é o container do Spring, por injeção via construtor.
- **Aberto/fechado.** Novas consultas entram estendendo o schema `.graphqls` e criando novos resolvers, sem alterar o que já existe. O mesmo vale para o tratamento de erros: uma nova exceção de negócio entra como um novo handler.
- **Segregação de interface.** Cada repositório declara somente as consultas que seu domínio precisa, em vez de uma interface de acesso a dados genérica e inflada.

DRY: eliminação de duplicação

- `FormatadorData` centraliza a formatação de `LocalDate` e `LocalDateTime`; nenhum DTO reimplementa o padrão de data.
- Cada DTO expõe uma factory `from(...)`, de modo que a conversão entidade para DTO exista em um único lugar.
- O `GlobalExceptionHandler` concentra o tratamento de erros da aplicação inteira, evitando blocos `try/catch` repetidos nas camadas.
- Nos testes, as classes abstratas `AbstractTest`, `AbstractControllerTest` e `AbstractHttpControllerTest` concentram as anotações comuns, e a query compartilhada vive em `QueryGraphQL`.

Convenções do Spring Boot

- **Injeção por construtor** com `@RequiredArgsConstructor` e campos `final`, o que deixa as dependências explícitas, imutáveis e fáceis de substituir em teste.
- **Transações declarativas:** `@Transactional(readOnly = true)` no serviço, alinhado à natureza somente-leitura da API.
- **Configuração externalizada** em `application.yaml` multi-documento, com um perfil por ambiente e credenciais vindas de variáveis de ambiente, sem nenhum segredo no código.
- **Estereótipos consistentes** (`@Controller`, `@Service`, `@Repository`, `@RestControllerAdvice`) e uso dos starters oficiais.
- **Logging via SLF4J** com `@Slf4j` e mensagens parametrizadas. O Log4j2 é a única implementação no classpath, com o Logback explicitamente excluído.

Modelagem, imutabilidade e camadas isoladas

- Os DTOs são `record` imutáveis, e o construtor compacto do `PacienteDTO` aplica `List.copyOf(...)` e substitui `null` por lista vazia, então a resposta nunca expõe coleção mutável nem obriga o cliente a tratar nulos.
- A entidade JPA jamais é serializada diretamente: o contrato público está isolado do mapeamento de banco, o que permite evoluir um sem quebrar o outro.
- Associações são `LAZY` e carregadas explicitamente por `@EntityGraph`, evitando tanto o problema N+1 quanto `LazyInitializationException`.

Tratamento de erros e segurança da informação

- O `GlobalExceptionHandler` mantém **dois conjuntos de handlers**, os `@ExceptionHandler` do Spring MVC e os `@GraphQLExceptionHandler`, já que o pipeline do GraphQL não passa pelo `HandlerExceptionResolver`.
- Mensagens de exceções que não são do domínio não voltam ao cliente: erros de integridade e falhas inesperadas são registrados no log e respondidos com texto fixo, sem vaziar nome de tabela, coluna ou constraint. A propriedade `include-stacktrace: never` impede o vazamento de stacktrace.

Testes automatizados e integração contínua

- **Testes de integração reais**, que sobem o contexto do Spring contra um PostgreSQL real, não um banco em memória.
- O resolver é exercitado de duas formas: pelo `GraphQLTester`, sem camada HTTP, e pelo transporte HTTP real, o que cobre também o context path.
- O Flyway está restrito ao classpath de teste: a suíte é autossuficiente e funciona sobre um banco vazio, inclusive na CI, sem permitir migração acidental em produção.
- A cada Pull Request para a `main`, o GitHub Actions provisiona um PostgreSQL 18, configura o Java 21 e executa `./gradlew build`; uma regressão reprova o PR antes do merge.

Repositório do Código

URL do repositório

- Repositório: <https://github.com/JavaPosTech/HistoricoAPI>
- Branch de entrega: `main`
- Versão: `1.0.0`

Organização do repositório

Caminho	Conteúdo
<code>/src/main/java</code>	Código-fonte da aplicação, organizado por camada e por domínio
<code>/src/main/resources/graphql</code>	Schemas GraphQL carregados pelo Spring for GraphQL
<code>/src/test</code>	Suíte de integração e migrations do perfil de teste
<code>docker-compose-postgres-dev.yml</code>	PostgreSQL de desenvolvimento e da suíte de testes
<code>docker-compose-postgres-prod.yml</code>	PostgreSQL de produção, com healthcheck e leitura do <code>.env</code>
<code>docker-compose-historicoapi.yml</code>	Orquestração da API no perfil prod
<code>Dockerfile</code>	Build multi-stage da imagem da aplicação
<code>.github/workflows</code>	Pipeline de integração contínua executada a cada Pull Request
<code>README.md</code>	Instruções resumidas de execução e documentação do projeto

Serviços relacionados na Fase 3

Serviço	Porta (host)	Papel
AgendamentoAPI	9027	Agendamento de consultas; dona das migrations Flyway da fase
HistoricoAPI	9028	Histórico de consultas, exposto por GraphQL; somente leitura